

PROGRAMMATION FONCTIONNELLE 2 : PARTIEL 1
mars 2026 — Pierre Rousselin

Exercice 1 : Preuves de logique propositionnelle de base, sur 12 points

Dans tout cet exercice, A, B et C sont trois propositions (en Rocq, des types de sorte Prop) quelconques. Dans chacun des cas suivants, donner :

- une preuve Rocq en programmation fonctionnelle, c'est-à-dire avec un seul terme de preuve dans `exact` ;
- une preuve Rocq avec des tactiques de plus haut niveau et `exact` seulement à la fin sur une hypothèse et
- un arbre de dérivation en déduction naturelle.

1. (4 points) `Lemma and_assoc (h : A /\ (B /\ C)) : (A /\ B) /\ C.`
2. (4 points) `Lemma or_comm (h : A \/ B) : B \/ A.`
3. (4 points) `Lemma imp_imp_and (h : A -> (B -> C)) : A /\ B -> C.`

--- * ---

Correction de l'exercice 1 :

1.a) `exact (match h with`
`| conj ha (conj hb hc) => conj (conj ha hb) hc`
`end).`

ou avec `let-pattern` :

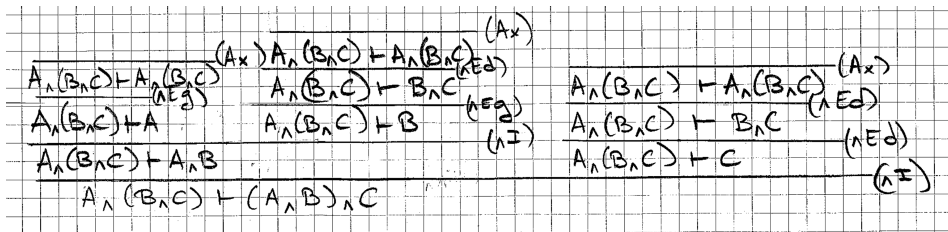
`exact (let '(conj ha (conj hb hc)) := h in conj (conj ha hb) hc).`

b) Version longue :

`destruct h as [ha hbc].`
`destruct hbc as [hb hc].`
`split.`
`- split.`
`+ exact ha.`
`+ exact hb.`
`- exact hc.`

Version courte :

`destruct h as [? [? ?]]; split; [split |]; assumption.`



c)

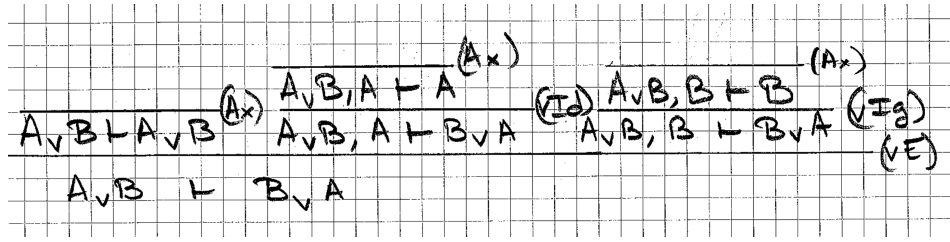
2.a) `exact (match h with`
`| or_introl hA => or_intror hA`
`| or_intror hB => or_introl hB`
`end).`

b) Version longue :

`destruct h as [ha | hb].`
`- right. exact ha.`
`- left. exact hb.`

Version courte :

`destruct h as [? | ?]; [right | left]; assumption.`



3.a) `exact (fun hab => match hab with
| conj ha hb => (h ha hb)
end).`

ou bien

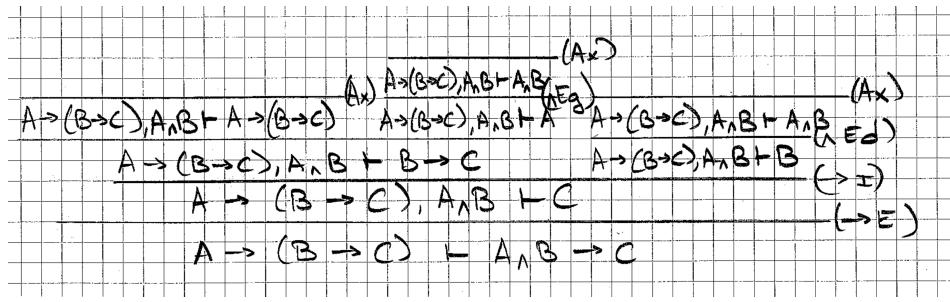
`exact (fun '(conj ha hb) => (h ha hb)).`

b) Version longue :

`intros hab.
destruct hab as [ha hb].
apply h.
- exact ha.
- exact hb.`

Version courte :

`intros [? ?]; apply h; assumption.`



--- * ---

Exercice 2 : Typage de fonctions, sur 13 points

On considère la fonction OCaml suivante :

`let dup f x = f x x`

On rappelle que la notation `( op )` en OCaml désigne la fonction (préfixe) associée à l'opérateur infixe `op`. Par exemple, `( + ) 3 2` vaut 5.

- (1 point) Quelle est la valeur de `dup ( + ) 3`? de `dup ( && ) true`?
- (1 point) Quelle est la valeur de `dup ( + )`? Comment appelle-t-on cette fonction en langage naturel?
- (1 point) Quelle est la valeur de `dup ( && )`? À quelle autre fonction plus simple est-elle égale?
- (1 point) Écrire une définition de `dup` sans aucun sucre syntaxique (avec le mot-clé `fun`).
- (2 points) Donner le type de `dup`.

On considère maintenant la fonction OCaml suivante :

`let const c x = c`

- (1 point) Quelle est la valeur de `const 42 true`?
- (1 point) Quelle est la valeur de `const (fun x -> x + 1) true 10`?

8. (1 point) Quel est le type de `const (fun x -> x + 1)` ?
9. (1 point) Quelle est la valeur de `const ( + ) true 4 2` ? de `const ( && ) 42 true false` ?
10. (1 point) Quel est le type de `const` ?
11. (1 point) Quelle est la valeur de `dup (const (fun x -> x + 1)) 42` ?
12. (1 point) Que dire (type et valeur) de `fun f -> dup (const f)` ?

--- * ---

Correction de l'exercice 2 :

1. `dup ( + ) 3 = ( + ) 3 3 = 6`
`dup ( && ) true = ( && ) true true = true`
2. `dup ( + )` vaut `fun x -> x + x` (doubler un entier)
3. `dup ( && )` vaut `fun x -> x && x` (extensionnellement égale à l'identité).
4. `let dup = fun f -> fun x -> f x x`
5. `dup : ('a -> 'a -> 'b) -> 'a -> 'b`
6. `const 42 true = 42`
7. `const (fun x -> x + 1) true 10 = (fun x -> x + 1) 10 = 11`
8. `const (fun x -> x + 1)` est de type `'a -> int -> int`
9. `const ( + ) true 4 2 = ( + ) 4 2 = 6`
`const ( && ) 42 true false = ( && ) true false = false`
10. `const : 'a -> 'b -> 'a`
11. `dup (const (fun x -> x + 1)) 42 = dup (fun z x -> x + 1) 42 = (fun y -> y + 1) 42 = 43`
12. `fun f -> dup (const f) = fun f -> dup (fun x -> f)`
`= fun f -> fun z -> ((fun x -> f) z z)`
`= fun f -> fun z -> f z`

qui est de type `('a -> 'b) -> 'a -> 'b` et est en fait l'identité sur les fonctions.

--- * ---

Exercice 3 : Entiers naturels, sur 13 points

La définition du type `nat` des entiers naturels en Rocq est rappelée en fin d'énoncé. On définit les fonctions `double` et `half` de la façon suivante :

```
Fixpoint double (n : nat) :=
  match n with
  | 0 => 0
  | S p => S (S (double p))
  end.
Fixpoint half (n : nat) :=
  match n with
  | 0 | 1 => 0
  | S (S p) => S (half p)
  end.
```

1. (1 point) Quelle est la preuve commune des 5 lemmes suivants ?
`Lemma half_0 : half 0 = 0.`
`Lemma half_1 : half 1 = 0.`
`Lemma half_succ_succ (n : nat) : half (S (S n)) = S (half n).`
`Lemma double_0 : double 0 = 0.`
`Lemma double_succ (n : nat) : double (S n) = S (S (double n)).`

2. (2 point) En justifiant chaque égalité avec un des lemmes ci-dessus, évaluer pas à pas :
 - a) `double 2` (c'est-à-dire `double (S (S O))`)
 - b) `half 5` (c'est-à-dire `half (S (S (S (S (S O))))`)
3. (2 point) Donner la preuve en Rocq, puis la « preuve papier très détaillée » (où chaque égalité est justifiée) du lemme suivant :


```
Lemma half_double (n : nat) : half (double n) = n.
```
4. (1 point) Que dire, pour un `n` de type `nat`, de `double (half n)`? (Une preuve formelle n'est pas demandée ici.)
5. (1 point) On rappelle les propriétés suivantes de l'addition :


```
Lemma add_0_l (n : nat) : 0 + n = n.
Lemma add_0_r (n : nat) : n + 0 = n.
Lemma add_succ_l (n m : nat) : S n + m = S (n + m).
Lemma add_succ_r (n m : nat) : n + S m = S (n + m).
```

 Parmi celles-ci, lesquelles ont besoin d'une preuve par induction?
6. (2 points) Donner la preuve en Rocq et la « preuve papier très détaillée » (où chaque égalité est justifiée) du lemme suivant :


```
Lemma double_eq_add_diag (n : nat) : double n = n + n.
```
7. (1 point) En déduire la preuve « papier très détaillée » de :


```
Lemma half_add_diag (n : nat) : half (n + n) = n.
```
8. (1 point) La multiplication des entiers naturelle est rappelée en fin d'énoncé. On considère l'énoncé suivant, avec un début de preuve :


```
Lemma twice_eq_add_diag (n : nat) : 2 * n = n + n.
Proof.
  simpl.
```

 Quel est le but après la tactique `simpl.`? Écrire les étapes successives de réduction. Donner la fin de la preuve en Rocq.
9. (1 point) Donner la définition en Rocq de la fonction `two_pow`, de `nat` vers `nat`, qui à un entier naturel n associe 2^n .
10. (1 point) Que dire, pour un `n` de type `nat` de `half (two_pow (S n))`? En donner une preuve papier très détaillée.

--- * ---

Correction de l'exercice 3 :

1. Ces lemmes sont tous conséquences immédiates, par calcul, des définitions. Donc on peut les prouver avec


```
simpl. reflexivity.
```

 (ou juste `reflexivity`).

2.a)

$$\begin{aligned} \text{double } 2 &= \text{double } (S (S O)) \stackrel{(\text{double_succ})}{=} S (S (\text{double } (S O))) \\ &\stackrel{(\text{double_succ})}{=} S (S (S (S (\text{double } O)))) \stackrel{(\text{double_0})}{=} S (S (S (S O))) = 4. \end{aligned}$$

b)

$$\begin{aligned} \text{half } 5 &= \text{half } (S (S (S (S (S O)))) \stackrel{(\text{half_succ_succ})}{=} S (\text{half } (S (S (S O)))) \\ &\stackrel{(\text{half_succ_succ})}{=} S (S (\text{half } (S O))) \stackrel{(\text{half_1})}{=} S (S O) = 2. \end{aligned}$$

3. En Rocq :

```

Lemma half_double (n : nat) : half (double n) = n.
Proof.
  induction n as [| p IH].
  - reflexivity.
  - rewrite double_succ half_succ_succ IH. reflexivity.
Qed.

```

Il est aussi faisable de remplacer des réécritures par du calcul avec `simpl`.

En « preuve papier très détaillée » :

On raisonne par induction sur n . Pour $n = 0$, on a :

$$\text{half (double 0)} \stackrel{(\text{double_0})}{=} \text{half0} \stackrel{(\text{half_0})}{=} .$$

Si $n = S p$, où p satisfait :

$$\text{half (double } p) = p, \quad (\text{IH})$$

alors

$$\begin{aligned} \text{half (double (S } p)) &\stackrel{(\text{double_succ})}{=} \text{half (S (S (double } p))} \\ &\stackrel{(\text{half_succ_succ})}{=} S (\text{half (double } p)) \stackrel{(\text{IH})}{=} S p. \end{aligned}$$

4. `double (half n)` est égal à n s'il est pair et à $n - 1$ s'il est impair, car `half` arrondit vers 0.
5. Les calculs sur l'opérande de gauche sont fait directement par réduction. Donc `add_0_1` et `add_succ_1` n'ont pas besoin d'induction, les deux autres lemmes, si.
6. Pour la preuve en Rocq :

```

Lemma double_eq_add_diag (n : nat) : double n = n + n.
Proof.
  induction n as [| n IH].
  - rewrite add_0_1 double_0. reflexivity.
  - rewrite double_succ IH add_succ_l add_succ_r.
    reflexivity.
Qed.

```

Là aussi, on peut laisser un peu `simpl` réduire. Pour la « preuve papier très détaillée » : On raisonne par induction sur n .

Si $n = 0$, alors, d'un côté,

$$\text{double 0} \stackrel{(\text{double_0})}{=} 0$$

et de l'autre côté,

$$0 + 0 \stackrel{(\text{add_0_1})}{=} 0.$$

Si $n = S p$, où p satisfait :

$$\text{double } p = p + p, \quad (\text{IH})$$

alors d'un côté,

$$\text{double (S } p) \stackrel{(\text{double_succ})}{=} S (\text{S (double } p)) \stackrel{(\text{IH})}{=} S (\text{S (} p + p))$$

De l'autre côté,

$$(S p) + (S p) \stackrel{(\text{add_succ_l})}{=} S (p + (S p)) \stackrel{(\text{add_succ_r})}{=} S (\text{S (} p + p))$$

7.

$$\text{half (n + n)} \stackrel{(\neg \text{double_eq_add_diag})}{=} \text{half (double } n) \stackrel{(\text{half_double})}{=} n.$$

8. Le terme $2 * n$ va être réduit de la façon suivante :

$$2 * n = S (S 0) * n = n + (S 0 * n) = n + (n + 0 * n) = n + (n + 0).$$

Ainsi, le but sera transformé en $n + (n + 0) = n + n$. On peut terminer la preuve avec

```
rewrite add_0_r.
reflexivity.
```

9. `Fixpoint two_pow (n : nat) :=`
`match n with`
`| 0 => 1`
`| S p => 2 * (two_pow p)`
`end.`

10. On a

$$\begin{aligned} \text{half (two_pow (Sn))} &= \text{half (2 * (two_pow n))} \\ &\stackrel{(\text{twice_eq_add_diag})}{=} \text{half ((two_pow n) + (two_pow n))} \\ &\stackrel{(\text{half_add_diag})}{=} \text{two_pow n.} \\ &\quad \text{--- * ---} \end{aligned}$$

Exercice 4 : Sur les listes, sur 10 points

La définition des listes est rappelée en fin d'énoncé, avec les fonctions `app` (notation : `l1 ++ l2` pour `app l1 l2`) et `length`. Dans tout l'exercice, `A` désigne un type quelconque. On considère la fonction `rev_append` définie ci-dessous :

```
Fixpoint rev_append (l1 l2 : list A) :=
  match l1 with
  | [] => l2
  | h :: t => rev_append t (h :: l2)
end.
```

Dans tout cet exercice, vous pourrez utiliser les lemmes de l'exercice précédent et les lemmes suivants (conséquences directes des définitions) :

```
Lemma app_nil_l (l : list A) : [] ++ l = l.
Lemma app_cons_l (h : A) (t l : list A) : (h :: t) ++ l = h :: (t ++ l).
Lemma length_nil : length (@nil A) = 0.
Lemma length_cons (h : A) (t : list A) : length (h :: t) = S (length t).
Lemma rev_append_nil_l (l : list A) : rev_append [] l = l.
Lemma rev_append_cons_l (h : A) (t l : list A) : rev_append (h :: t) l = rev_append t (h :: l).
```

- (1 point) Évaluer en donnant toutes les étapes de calcul et en justifiant chaque égalité : `rev_append [3; 4] [1; 2]`.
- (2 points) Écrire la preuve papier très détaillée de :
`Lemma length_app (l1 l2 : list A) : length (l1 ++ l2) = length l1 + length l2.`
- (2 points) En raisonnant par induction sur `l1`, **tout en gardant `l2` quelconque** (l'hypothèse d'induction est donc encore universellement quantifiée), écrire la preuve papier très détaillée de :
`Lemma length_rev_append (l1 : list A) :`
`forall l2, length (rev_append l1 l2) = length l1 + length l2.`
- (2 points) Même chose pour :
`Lemma rev_append_rev_append (l1 : list A) :`
`forall (l2 l3 : list A), rev_append (rev_append l1 l2) l3 = rev_append l2 (l1 ++ l3).`
- (1 point) Définir la fonction `rev`, qui renverse une liste, en utilisant `rev_append`.
- (1 point) Prouver avec une preuve papier très détaillée :

Lemma `rev_rev` $(l : \text{list } A) : \text{rev } (\text{rev } l) = l.$

7. (1 point) En déduire :

Lemma `rev_inj` $(l_1 l_2 : \text{list } A) : \text{rev } l_1 = \text{rev } l_2 \rightarrow l_1 = l_2.$

--- * ---

Correction de l'exercice 4 :

1.

$$\begin{aligned} \text{rev_append } [3; 4] [1; 2] &\stackrel{(\text{rev_append_cons_l})}{=} \text{rev_append } [4] [3; 1; 2] \\ &\stackrel{(\text{rev_append_cons_l})}{=} \text{rev_append } [] [4; 3; 1; 2] \\ &\stackrel{(\text{rev_append_nil_l})}{=} [4; 3; 1; 2]. \end{aligned}$$

2. On raisonne par induction sur l_1 .

Si $l_1 = []$, alors d'un côté,

$$\text{length } ([] ++ l_2) \stackrel{(\text{app_nil_l})}{=} \text{length } l_2$$

De l'autre côté,

$$\text{length } [] + \text{length } l_2 \stackrel{(\text{length_nil})}{=} 0 + \text{length } l_2 \stackrel{(\text{add_0_l})}{=} \text{length } l_2.$$

On suppose maintenant que $l_1 = h :: t$ pour un élément h de type A et t de type $\text{list } A$ satisfaisant :

$$\text{length } (t ++ l_2) = (\text{length } t) + (\text{length } l_2). \quad (\text{IH})$$

Alors, d'un côté on a :

$$\begin{aligned} \text{length } ((h :: t) ++ l_2) &\stackrel{(\text{app_cons_l})}{=} \text{length } (h :: (t ++ l_2)) \\ &\stackrel{(\text{length_cons})}{=} S (\text{length } (t ++ l_2)) \stackrel{(\text{IH})}{=} S (\text{length } t + \text{length } l_2), \end{aligned}$$

de l'autre, on a :

$$\begin{aligned} \text{length } (h :: t) + \text{length } l_2 &\stackrel{(\text{length_cons})}{=} (S (\text{length } t)) + \text{length } l_2 \\ &\stackrel{(\text{add_succ_l})}{=} S (\text{length } t + \text{length } l_2). \end{aligned}$$

3. On raisonne par induction sur l_1 .

Cas de base. On doit prouver :

$$\forall l_2, \text{length } (\text{rev_append } [] l_2) = \text{length } [] + \text{length } l_2 \quad (\text{but-cas-de-base})$$

Soit l_2 un élément de type $\text{list } A$. On a d'un côté,

$$\text{length } (\text{rev_append } [] l_2) \stackrel{(\text{rev_append_nil_l})}{=} \text{length } l_2$$

De l'autre côté,

$$\text{length } [] + \text{length } l_2 \stackrel{(\text{length_nil})}{=} 0 + \text{length } l_2 \stackrel{(\text{add_0_l})}{=} \text{length } l_2.$$

On suppose maintenant que $l_1 = h :: t$, où t satisfait :

$$\forall l, \text{length } (\text{rev_append } [] l) = \text{length } [] + \text{length } l \quad (\text{IH})$$

Soit l_2 une liste. On a d'un côté :

$$\begin{aligned} \text{length } (\text{rev_append } (h :: t) l_2) & \stackrel{(\text{rev_append_cons_l})}{=} \text{length } (\text{rev_append } t (h :: l_2)) \\ & \stackrel{(\text{IH avec } l \leftarrow h :: l_2)}{=} \text{length } t + \text{length } (h :: l_2) \\ & \stackrel{(\text{length_cons})}{=} \text{length } t + S (\text{length } l_2) \\ & \stackrel{(\text{add_succ_r})}{=} S (\text{length } t + \text{length } l_2). \end{aligned}$$

De l'autre côté,

$$\text{length } (h :: t) + \text{length } l_2 \stackrel{(\text{length_cons})}{=} S (\text{length } t) + \text{length } l_2 \stackrel{(\text{add_succ_l})}{=} S (\text{length } t + \text{length } l_2).$$

4. On raisonne par induction sur l_1 .

Cas de base. Soient l_2 et l_3 deux listes quelconques. On a d'un côté :

$$\text{rev_append } (\text{rev_append } [] l_2) l_3 \stackrel{(\text{rev_append_nil_l})}{=} \text{rev_append } l_2 l_3$$

De l'autre côté,

$$\text{rev_append } l_2 ([] ++ l_3) \stackrel{(\text{app_nil_l})}{=} \text{rev_append } l_2 l_3.$$

On suppose maintenant que $l_1 = h :: t$, où t satisfait :

$$\forall l_2, \forall l_3, \text{rev_append } (\text{rev_append } t l_2) l_3 = \text{rev_append } l_2 (t ++ l_3). \quad (\text{IH})$$

Soient l_2 et l_3 deux listes quelconques. On a d'un côté :

$$\begin{aligned} \text{rev_append } (\text{rev_append } (h :: t) l_2) l_3 & \stackrel{(\text{rev_append_cons_l})}{=} \text{rev_append } (\text{rev_append } t (h :: l_2)) l_3 \\ & \stackrel{(\text{IH avec } l_2 \leftarrow h :: l_2)}{=} \text{rev_append } (h :: l_2) (t ++ l_3) \\ & \stackrel{(\text{rev_append_cons_l})}{=} \text{rev_append } l_2 (h :: (t ++ l_3)) \end{aligned}$$

De l'autre côté,

$$\text{rev_append } l_2 ((h :: t) ++ l_3) \stackrel{(\text{app_cons_l})}{=} \text{rev_append } l_2 (h :: (t ++ l_3)).$$

5. **Definition** $\text{rev } (l : \text{list } A) := \text{rev_append } l []$.

6.

$$\begin{aligned} \text{rev } (\text{rev } l) & \stackrel{(\text{Par définition de rev})}{=} \text{rev_append } (\text{rev_append } l []) [] \\ & \stackrel{(\text{rev_append_rev_append})}{=} \text{rev_append } [] (l ++ []) \\ & \stackrel{(\text{rev_append_nil_l})}{=} l ++ [] \\ & \stackrel{(\text{app_nil_r})}{=} l \end{aligned}$$

À condition d'avoir `app_nil_r`, qui a été malheureusement oublié dans l'énoncé... mais se prouve très facilement par induction.

7. Soient l_1 et l_2 deux listes telles que

$$\text{rev } l_1 = \text{rev } l_2.H \quad (1)$$

Alors,

$$l_1 \stackrel{(\text{-rev_rev})}{=} \text{rev } (\text{rev } l_1) \stackrel{(H)}{=} \text{rev } (\text{rev } l_2) \stackrel{(\text{rev_rev})}{=} l_2.$$

--- *

Annexe 1 : règles de la déduction naturelle

$$\begin{array}{c}
 \frac{}{\Gamma, A \vdash A} (\text{Ax}) \\
 \frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \wedge B} (\wedge\text{I}) \qquad \frac{\Gamma \vdash A \wedge B}{\Gamma \vdash A} (\wedge\text{E-g}) \qquad \frac{\Gamma \vdash A \wedge B}{\Gamma \vdash B} (\wedge\text{E-d}) \\
 \frac{\Gamma \vdash A}{\Gamma \vdash A \vee B} (\vee\text{I-g}) \qquad \frac{\Gamma \vdash B}{\Gamma \vdash A \vee B} (\vee\text{I-d}) \qquad \frac{\Gamma \vdash A \vee B \quad \Gamma, A \vdash P \quad \Gamma, B \vdash P}{\Gamma \vdash P} (\vee\text{E}) \\
 \frac{\Gamma, A \vdash B}{\Gamma \vdash A \rightarrow B} (\rightarrow\text{I}) \qquad \frac{\Gamma \vdash A \rightarrow B \quad \Gamma \vdash A}{\Gamma \vdash B} (\rightarrow\text{E}) \\
 \frac{\Gamma, x : T \vdash P}{\Gamma \vdash \forall x : T, P} (\forall\text{I}) \qquad \frac{\Gamma \vdash \forall (x : T), P \quad \Gamma \vdash t : T}{\Gamma \vdash P\{x/t\}} (\forall\text{E})
 \end{array}$$

Quelques définitions standard en Rocq

```

Inductive bool := true | false.
Inductive and (A B : Prop) : Prop := conj (hA : A) (hB : B).
Inductive or (A B : Prop) : Prop :=
  or_introl (hA : A) | or_intror (hB : B).
Inductive nat : Set := 0 : nat | S : nat -> nat.
(* Notations usuelles : 0 pour 0, 1 pour (S 0), etc *)
Fixpoint add (n m : nat) :=
  match n with
  | 0 => m
  | S p => S (add p m)
  end.
Fixpoint mul (n m : nat) :=
  match n with
  | 0 => 0
  | S p => m + p * m
  end.
Inductive list (A : Type) : Type :=
  nil : list A | cons : A -> list A -> list A.
(* Notation [] pour nil et h :: t pour (cons h t) *)
Fixpoint app {A : Type} (l0 l1 : list A) :=
  match l0 with
  | [] => l1
  | h :: t => h :: (app t l1)
  end.
Fixpoint length {A : Type} (l : list A) :=
  match l with
  | [] => 0
  | _ :: t => S (length t)
  end.

```

Liste de tactiques Rocq vues en cours, TD et TP

- **exact** : terminer la preuve en cours avec un terme de preuve
- **destruct** : raisonner par cas sur une valeur en liant des variables éventuelles aux arguments éventuels du ou des constructeurs
- **left** ou **right** : choisir le premier ou le deuxième constructeur ; plus souvent utilisé pour prouver une disjonction

- **split** : utiliser l'unique constructeur ; plus souvent utilisé pour prouver une disjonction
- **reflexivity** : terminer la preuve en cours lorsque le but se réduit à $a = a$
- **rewrite** : remplacer un terme par un autre qui lui est égal
- **induction** : comme **destruct** mais avec des hypothèses d'induction dans le(s) cas d'argument(s) dont le type est le même que la valeur sur laquelle on raisonne par induction
- **specialize** : donner des arguments à une fonction ou, ce qui revient au même, instancier un lemme.
- **simpl** : calculer en utilisant les définitions
- **intros** : introduire des variables ou hypothèses depuis le but vers le contexte local